

ECE 220 Computer Systems & Programming

Lecture 4 — Programming with the Stack

September 3, 2026

Prof. Sayan Mitra `mitras@illinois.edu`

Course website: `courses.grainger.illinois.edu/ece220/fa2026`

I ILLINOIS

Electrical & Computer Engineering

GRAINGER COLLEGE OF ENGINEERING

Two Notations for Expressions: Infix and Postfix

Infix — what you type

$$7 + (9 - 6) / 3$$

- operator sits *between* its operands
- needs **parentheses** and precedence rules (“/ before +”)
- without them the meaning changes:
 $7 + 9 - 6 / 3 \neq$ the above

Two Notations for Expressions: Infix and Postfix

Infix — what you type

$$7 + (9 - 6) / 3$$

- operator sits *between* its operands
- needs **parentheses** and precedence rules (“/ before +”)
- without them the meaning changes:
 $7 + 9 - 6 / 3 \neq$ the above

Postfix — operator *after* its operands

$$7 \ 9 \ 6 \ - \ 3 \ / \ +$$

- each operator applies to the two values written just before it
- no parentheses — ever**
- unambiguous: one reading, left to right

$$\begin{array}{l} 9 - 6 \longrightarrow \underline{\hspace{2cm}} \\ (9 - 6) / 3 \longrightarrow \underline{\hspace{2cm}} \\ 7 + (9 - 6) / 3 \longrightarrow \underline{\hspace{2cm}} \end{array}$$

Both = 8. How do we *evaluate* $7 \ 9 \ 6 \ - \ 3 \ / \ +$, reading left to right?

Today's Two Questions

Parentheses are just pushes and pops in disguise.

1. How does a stack evaluate *any* arithmetic expression — without parentheses?
2. What must stack programs do when things go wrong — empty pops, full pushes, bad input?

Applications today: palindromes, balanced parentheses, and a working calculator core — the heart of MP2.

Warm-up: Which Stack Is Empty?

Recall our convention: **TOP (R6) points at the top element**; the stack is empty when $TOP = STACK_START$.

STACK_END	x3FFB	/////	
	x3FFC	x39	
	x3FFD	x0	
	x3FFE	x3	
	x3FFF	x40	
STACK_START	x4000		← TOP

(a)

STACK_END	x3FFB	x0	
	x3FFC	x0	
	x3FFD	x0	← TOP
	x3FFE	x0	
	x3FFF	x0	
STACK_START	x4000		

(b)

Which one is empty? _____

Review: PUSH and POP — Just the Contract

Two subroutines from Lecture 3. Today we use them as **library routines** — all that matters is what they promise:

PUSH

- **IN:** R0 = value to push
- **OUT:** R5 = 0 (success) or 1 (**overflow**: stack full)
- Every other register: unchanged

POP

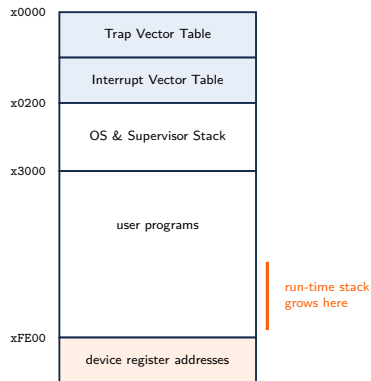
- **IN:** nothing
- **OUT:** R0 = the popped value;
R5 = 0 (success) or 1 (**underflow**: stack empty)
- Every other register: unchanged

Calling them is a JSR — so a subroutine that uses PUSH/POP must save its own R7 first.

How they work inside (TOP pointer, boundary checks) was Lecture 3; today it stays behind the curtain.

The Run-Time Stack

- Each invoked subroutine gets a memory template called an **activation record (stack frame)**
- Activation records are **pushed** onto the **run-time stack** in the order the calls happen — and popped in reverse order, exactly LIFO
- This is how every language you will ever use implements function calls
- The **supervisor stack** (OS) is separate — details at the end of the semester



Palindrome Check Using a Stack

A sequence that **reads the same forward and backward**:

`madam` `kayak` *Was it a car or a cat I saw* `123456654321`
`aibohphobia` (the fear of palindromes — an affliction its sufferers cannot even name)

How can a stack decide this?

- _____ of the characters onto the stack
- Then, for each character of the second half: _____
- Palindrome $\Leftrightarrow$ every comparison matches

In-Class Problem: Palindrome in LC-3

R1 - scan pointer into STR
 R0 - PUSH input / POP output
 R2 - expected char (rescan)
 R5 - 0 palindrome, 1 not
 R7 - saved: we JSR inside!

```

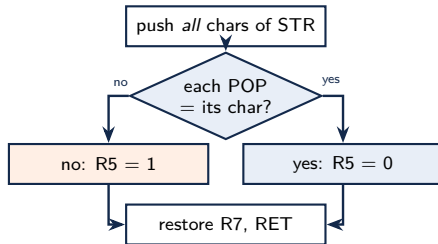
STR      .STRINGZ "madam"
PALIN    ST      R7, PL_SAVER7 ; we JSR below!
        ; pass 1: push every char of STR (stop at the NUL)
        PL_PUSH
  
```

```

        ; pass 2: rescan; each POP must equal the char
        PL_CMP   LEA   R1, STR
        PL_LOOP
  
```

```

PL_YES   AND     R5, R5, #0      ; R5 = 0: yes
        BR      PL_DONE
PL_NO    AND     R5, R5, #0
        ADD     R5, R5, #1      ; R5 = 1: no
PL_DONE  LD      R7, PL_SAVER7
        RET
  
```



Why can POP never underflow here?

Balanced Parentheses Using a Stack

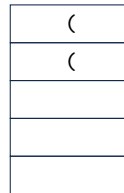
Balanced: `((()()()()))` `(((((()))))`

Unbalanced: `(((((())((` `(((((())` `((()((`

- Open parenthesis (: _____ onto the stack
- Close parenthesis) : _____ from the stack

Unbalanced is detected two ways:

1. At the end of the expression: _____
2. While scanning: _____



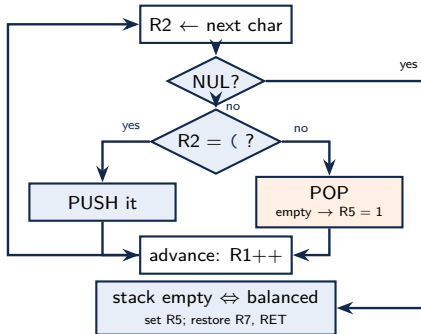
one push per (

Balanced Parentheses: the Stack in Action

R1 - scan pointer into PSTR
 R2 - current char
 R3 - compares: -'(', TOP-START
 R0 - PUSH input / POP output
 R5 - 0 balanced (POP supplies the 1)
 R7 - saved: we JSR inside!

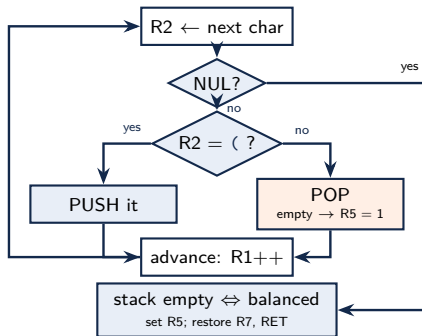
PSTR .STRINGZ "(()())"

Trace it: draw the stack after *each* character.



In-Class Problem: Balanced Parentheses in LC-3

R1 - scan pointer into PSTR
 R2 - current char
 R3 - compares: -'(', TOP-START
 R0 - PUSH input / POP output
 R5 - 0 balanced (POP supplies the 1)
 R7 - saved: we JSR inside!



```

PSTR    .STRINGZ "(()())"
BALCK   ST    R7, B_SAVER7    ; we JSR below!
        LEA   R1, PSTR
B_LOOP  LDR   R2, R1, #0      ; next char
        BRz   B_END          ; NUL: string done
        ; is it '(' ? if not -> B_POP (hint: NEG_OPEN)
  
```

```

        ADD   R0, R2, #0      ; '(' : push it
        JSR   PUSH
        BR    B_NEXT
B_POP   ; ')' : pop --- POP's underflow R5 is our answer!
  
```

```

B_NEXT  ADD   R1, R1, #1
        BR    B_LOOP
B_END   ; balanced iff the stack is empty (hint: NEG_START)
  
```

```

B_DONE  LD    R7, B_SAVER7
        RET
  
```

Postfix Expressions (Single-Digit Operands)

Infix	Postfix
$(3+4)-5$	$34+5-$
$2^{(8-4)}$	_____
$7+(9-6)/3$	_____
_____	$512+4*+3-$

Note: $12-$ means $1 - 2$, not $2 - 1$.

Are these valid postfix expressions? How would your program know?

- $46*-$ _____
- $13+57$ _____

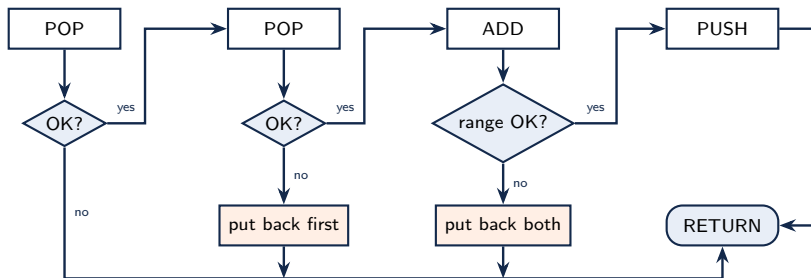
In-Class Trace: Evaluate $34+5-$

Token	Action	Stack (top last)
3	push 3	_____
4	push 4	_____
+	pop 4, pop 3, push $3 + 4$	_____
5	push 5	_____
-	pop 5, pop 7, push $7 - 5$	_____

- One value left at the end — that is the answer: _____
- For $-$ and $/$: the **first** pop is the **right** operand

Arithmetic Using a Stack: the ADD Subroutine

Pop two numbers, add them, push the result — *robustly*:



Given (callee-saved): **PUSH** (IN R0; OUT R5), **POP** (OUT R0, R5), **CHECK_RANGE** (IN R0; OUT R5 = 0 iff $-100 \leq R0 \leq 100$). **What must ADD watch out for?**

In-Class Problem: Implement ADD

R1 - 1st operand (popped) R2 - 2nd operand R0 - PUSH/POP data, the sum R5 - status R7 - saved: we JSR!

Success path:

```
ADDSUB ST R1, A_SAVER1
      ST R2, A_SAVER2
      ST R7, A_SAVER7 ; we call subroutines!
; pop first operand; on failure (R5=1) -> A_EXIT
```

```
; save it in R1
```

```
; pop second; on failure -> A_PUT1
```

```
; save it in R2; compute the sum in R0
```

```
; check range; on failure -> A_PUT2
```

```
; push the sum; then BR to A_EXIT
```

Failure paths (given):

```
A_PUT2 ADD R0, R2, #0 ; second first...
      JSR PUSH
A_PUT1 ADD R0, R1, #0 ; ...then first
      JSR PUSH
      AND R5, R5, #0
      ADD R5, R5, #1 ; report failure
A_EXIT LD R1, A_SAVER1
      LD R2, A_SAVER2
      LD R7, A_SAVER7
      RET
```

Why push the *second* operand back first?

Note the ST R7 — ADD calls subroutines, so it must save its own way home.

Summary & What's Next

Today

- Only the **TOP pointer** defines the stack — contents are irrelevant (the warm-up); PUSH/POP from Lecture 3 are today's building blocks
- The **run-time stack** holds one activation record per live call — LIFO is exactly the shape of nested calls
- Stacks solve real problems: palindromes, balanced parentheses, and **postfix evaluation** — push operands; operators pop, compute, push
- Robust stack code checks *every* pop and push, and leaves the stack unchanged on failure

Next lecture

- You now know everything the LC-3 offers: registers, memory, TRAPs, subroutines, the stack. Next: a language that manages them for you — **C**

Code from today: `palin.asm`, `balanced.asm`, and `stack_add.asm` (each with a `_skel` version) in the course code repository.